

ULTIMATE NEXT JS COURSE

WITH PROJECT





Prerequisites for this video

VIRTUAL CODE

- HTML & CSS
- Java Script
- React Js
- Mongo DB



ULTIMATE NEXT JS COURSE



LEVEL 1

INTRODUCTION TO NEXT JS



WHAT IS NEXT JS ?

- **Next.js is a React-based open-source web framework that enables developers to build fast, scalable, and SEO-friendly web applications with features like server-side rendering (SSR), static site generation (SSG), API routes, and built-in optimization for performance and routing.**
- ✨ In short: *It's React, but with superpowers like SSR, SSG, routing, and full-stack capabilities.*

```
npx create-next-app@latest
```


ULTIMATE NEXT JS COURSE



LEVEL 2

App Router



Introduction to App Router -

- Next.js uses file-system routing, which means the routes in your application are determined by how you structure your files.





INTRODUCTION TO NEXT JS

VIRTUAL CODE





Nested Routes-

folder

Route segment

folder/folder

Nested route segment



Dynamic Routes-

`[folder]`

Dynamic route segment

`[...folder]`

Catch-all route segment

`[...folder]`

Optional catch-all route segment



Dynamic Route Segment-

- A Dynamic Segment can be created by wrapping a folder's name in square brackets: [folderName]. For example, a blog could include the following route `app/blog/[slug]/page.js` where [slug] is the Dynamic Segment for blog posts.
- Since the `params` prop is a promise. You must use `async/await` or React's `use` function to access the values.

`js` `app/blog/[slug]/page.js`

JavaScript



```
export default async function Page({ params }) {  
  const { slug } = await params  
  return <div>My Post: {slug}</div>  
}
```



Catch-all Segments

Dynamic Segments can be extended to **catch-all** subsequent segments by adding an ellipsis inside the brackets [...folderName].

For example, `app/shop/[...slug]/page.js` will match `/shop/clothes`, but also `/shop/clothes/tops`, `/shop/clothes/tops/t-shirts`, and so on.

Route	Example URL	params
<code>app/shop/[... slug]/page.js</code>	<code>/shop/a</code>	<code>{ slug: ['a'] }</code>
<code>app/shop/[... slug]/page.js</code>	<code>/shop/a/b</code>	<code>{ slug: ['a', 'b'] }</code>
<code>app/shop/[... slug]/page.js</code>	<code>/shop/a/b/c</code>	<code>{ slug: ['a', 'b', 'c'] }</code>



Optional Catch-all Segments

Catch-all Segments can be made **optional** by including the parameter in double square brackets: `[...folderName]`.

For example, `app/shop/[...slug]/page.js` will **also** match `/shop`, in addition

to `/shop/clothes`, `/shop/clothes/tops`, `/shop/clothes/tops/t-shirts`.

The difference between **catch-all** and **optional catch-all** segments is that with optional, the route without the parameter is also matched (`/shop` in the example above).



Optional Catch-all Segments

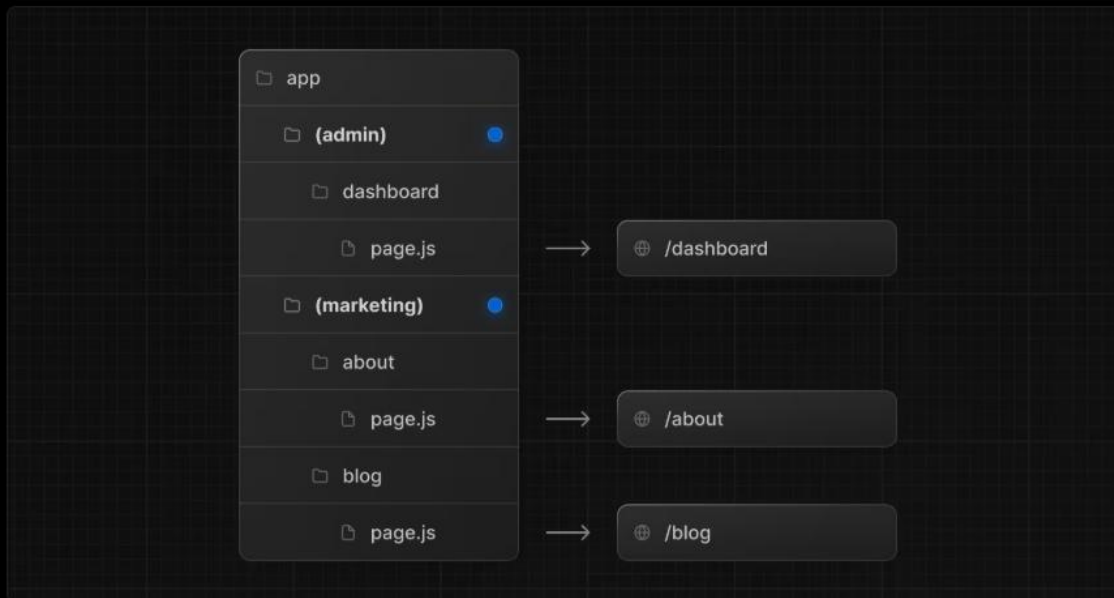
Route	Example URL	params
<code>app/shop/[[... slug]]/page.js</code>	<code>/shop</code>	<code>{ slug: undefined }</code>
<code>app/shop/[[... slug]]/page.js</code>	<code>/shop/a</code>	<code>{ slug: ['a'] }</code>
<code>app/shop/[[... slug]]/page.js</code>	<code>/shop/a/b</code>	<code>{ slug: ['a', 'b'] }</code>
<code>app/shop/[[... slug]]/page.js</code>	<code>/shop/a/b/c</code>	<code>{ slug: ['a', 'b', 'c'] }</code>



Routes Group -

A route group can be created by wrapping a folder's name in parenthesis: `(folderName)`.

This convention indicates the folder is for organizational purposes and should **not be included** in the route's URL path.





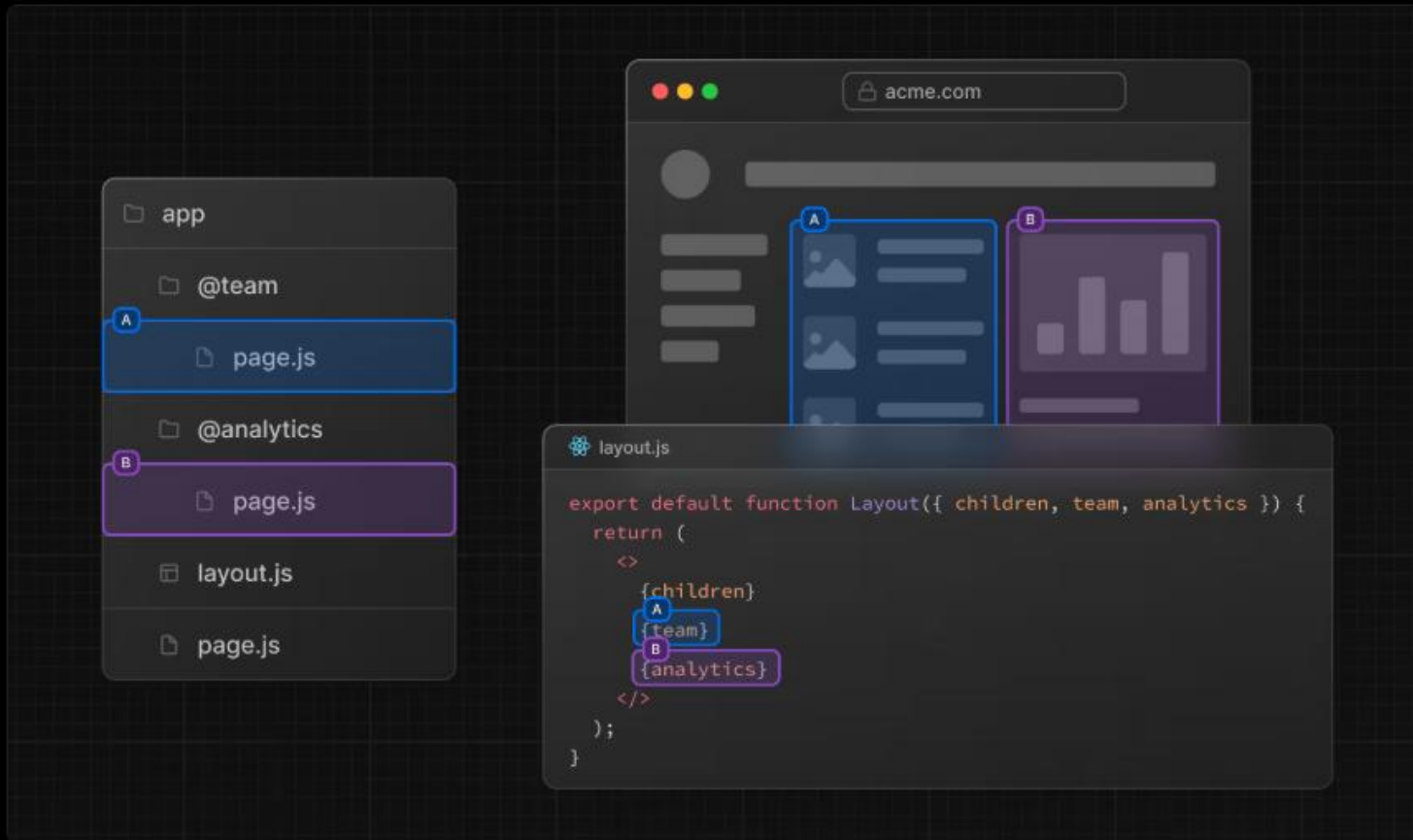
Parallel Routes -

Parallel Routes allows you to simultaneously or conditionally render one or more pages within the same layout. They are useful for highly dynamic sections of an app, such as dashboards and feeds on social sites.

For example, considering a dashboard, you can use parallel routes to simultaneously render the `team` and `analytics` pages:



Parallel Routes -





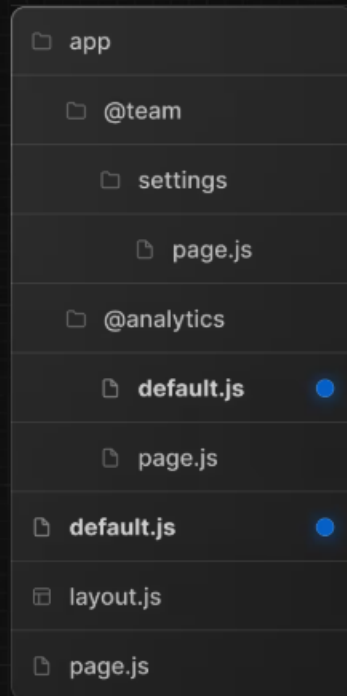
App Router

VIRTUAL CODE

default.js

You can define a `default.js` file to render as a fallback for unmatched slots during the initial load or full-page reload.

Consider the following folder structure. The `@team` slot has a `/settings` page, but `@analytics` does not.





Routing Files

layout	.js .jsx .tsx	Layout
page	.js .jsx .tsx	Page
loading	.js .jsx .tsx	Loading UI
not-found	.js .jsx .tsx	Not found UI
error	.js .jsx .tsx	Error UI
global-error	.js .jsx .tsx	Global error UI
route	.js .ts	API endpoint
template	.js .jsx .tsx	Re-rendered layout
default	.js .jsx .tsx	Parallel route fallback page



Note -

In next js , every page is server component so if you want to use hooks (useState.... etc) , you have to write 'use client' at the top of the page.

ULTIMATE NEXT JS COURSE



LEVEL 3

Navigation , Image & Font in Next JS



Navigation in Next JS -

There are two methods of Navigation in Next JS

1. **Declarative navigation** → `<Link>` component ke through
2. **Programmatic navigation** → `useRouter()` hook ke through



◆ 1. Declarative Navigation (using `<Link>`)

Ye simple aur recommended method hai — directly UI me link likh dete ho:

jsx

 Copy code

```
import Link from 'next/link'

export default function Home() {
  return (
    <div>
      <h1>Home Page</h1>
      <Link href="/about">Go to About Page</Link>
    </div>
  )
}
```

✓ **Benefit:** Fast client-side routing, automatic prefetching, SEO-friendly.



◆ 2. Programmatic Navigation (using `useRouter`)

Kabhi-kabhi hume navigation **automatically** karni hoti hai — jaise:

- Form submit hone ke baad next page par redirect karna,
- Button click se navigate karna (without Link).

Tab hum `useRouter()` hook use karte hain.



Navigation , Image & Font in Next JS

VIRTUAL CODE

```
jsx

'use client'
import { useRouter } from 'next/navigation'

export default function Home() {
  const router = useRouter()

  const goToAbout = () => {
    router.push('/about')
  }

  return (
    <div>
      <h1>Home Page</h1>
      <button onClick={goToAbout}>Go to About</button>
    </div>
  )
}
```



Image Optimization -

Automatically resizes, compresses, and serves images in modern formats (like WebP/AVIF) for faster loading and better performance.

```
import Image from 'next/image'

export default function Home() {
  return (
    <Image
      src="/images/sample.jpg"
      alt="Sample Image"
      width={600}
      height={400}
      priority
    />
  )
}
```



Font Optimization -

Reduces font file size by including only needed characters (subsets) and preloading them for faster page rendering and minimal layout shifts.

```
import { Roboto } from 'next/font/google'

const roboto = Roboto({ subsets: ['latin'], weight: ['400', '700'], display: 'swap' })

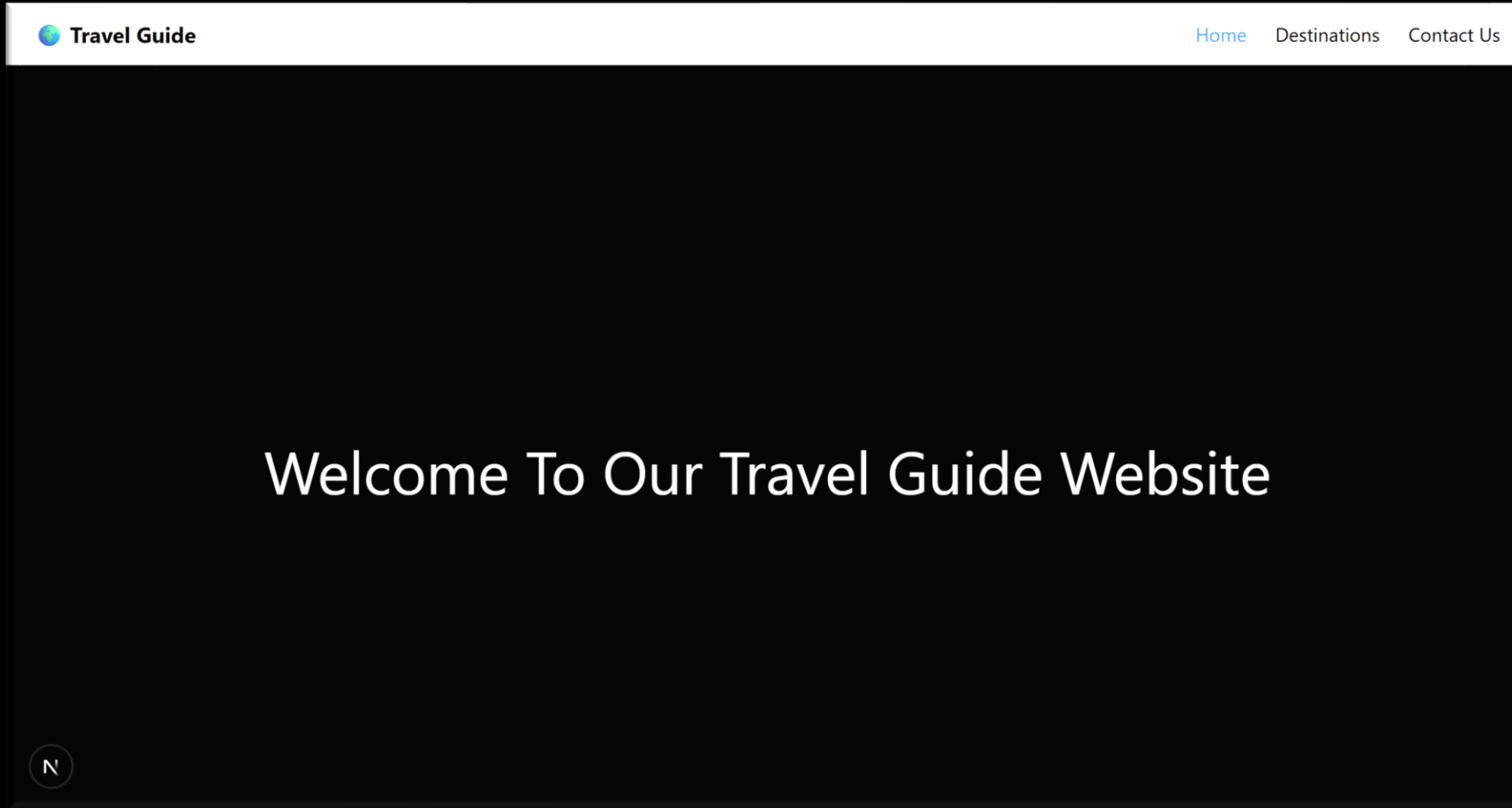
export default function Home() {
  return <h1 className={roboto.className}>Hello, Next.js Fonts!</h1>
}
```



Practice Project 1

VIRTUAL CODE

Build a simple Travel Guide website using next JS -

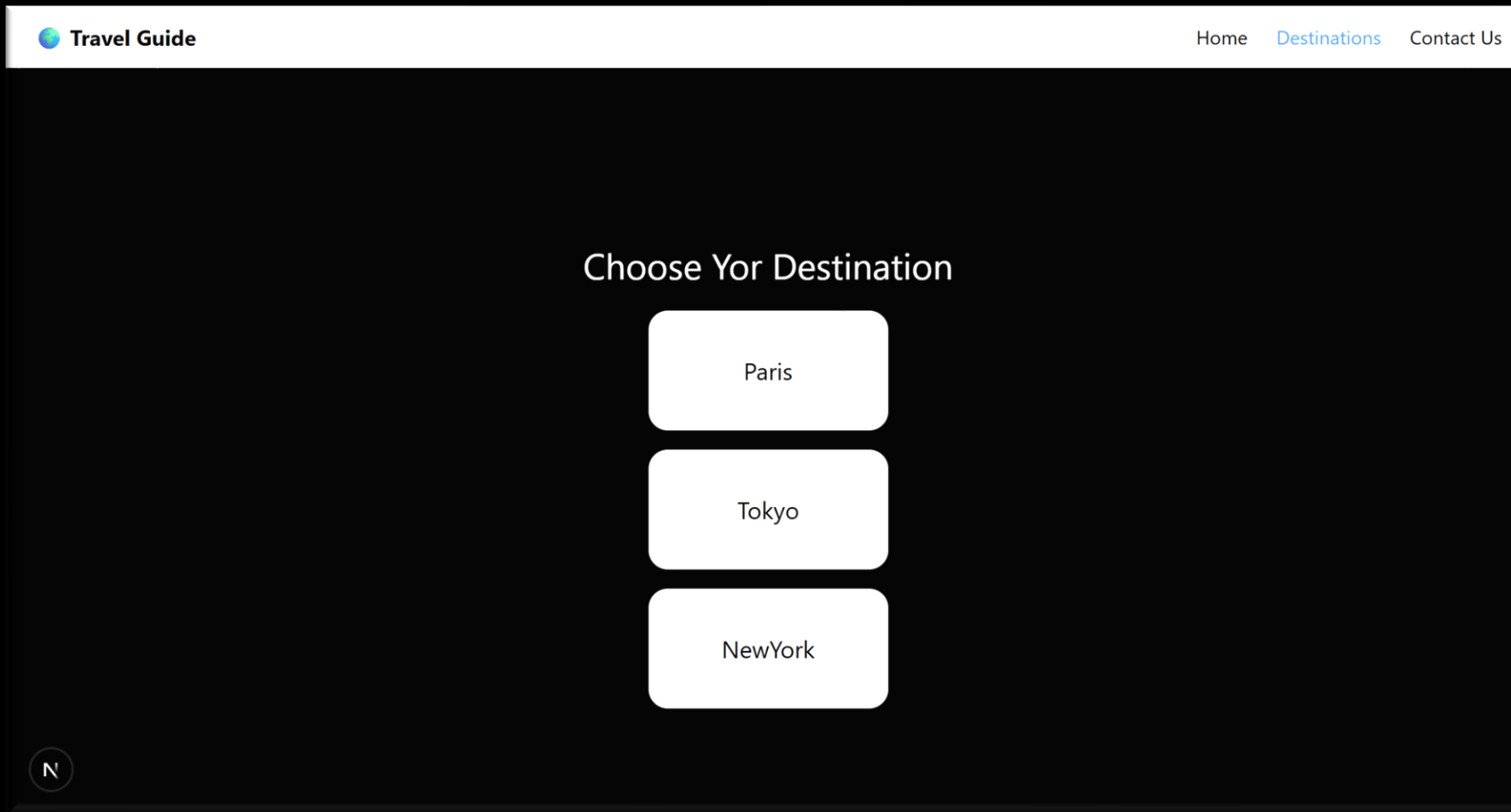




Practice Project 1

VIRTUAL CODE

Build a simple Travel Guide website using next JS -

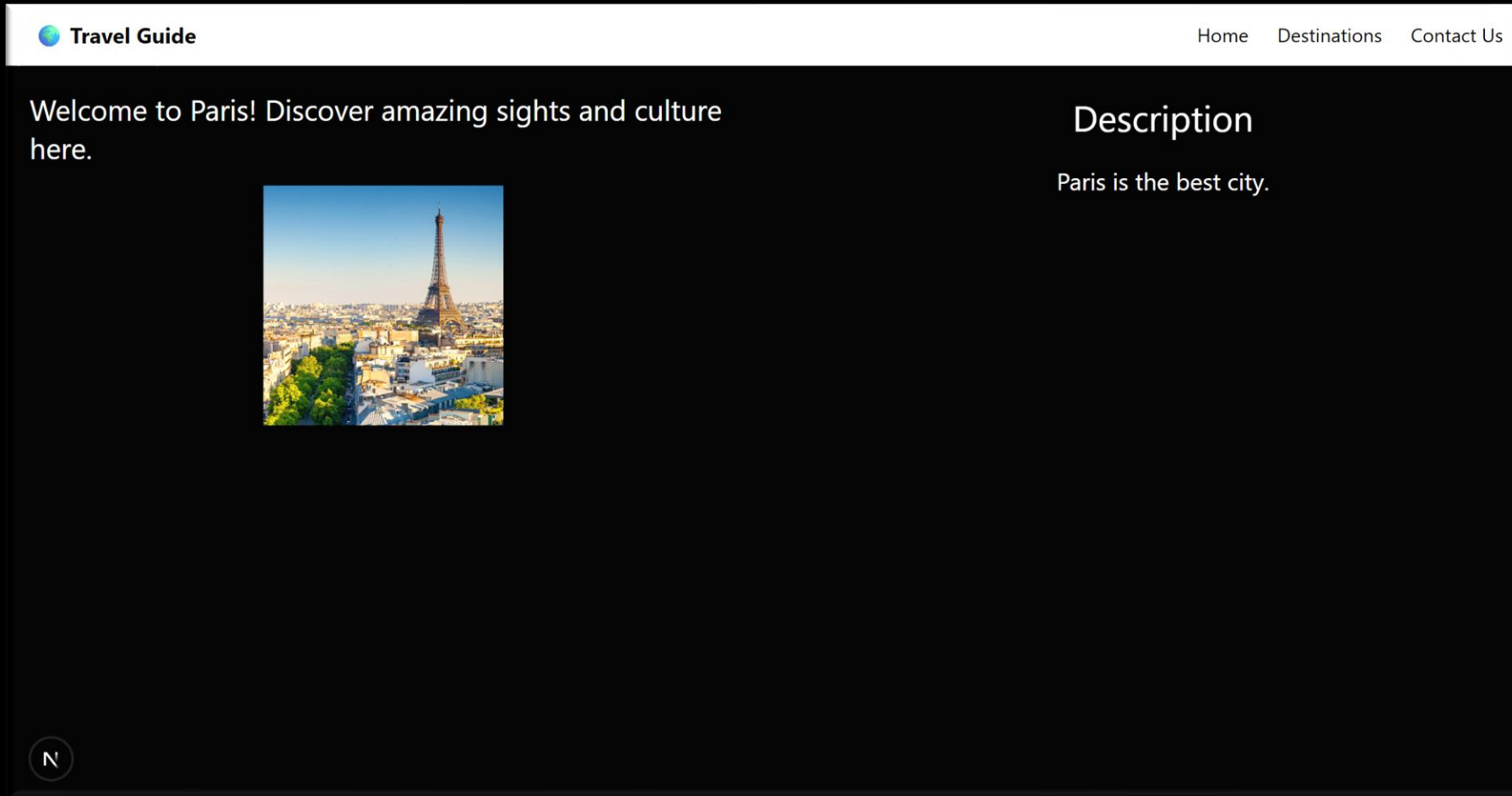




Practice Project 1

VIRTUAL CODE

Build a simple Travel Guide website using next JS -

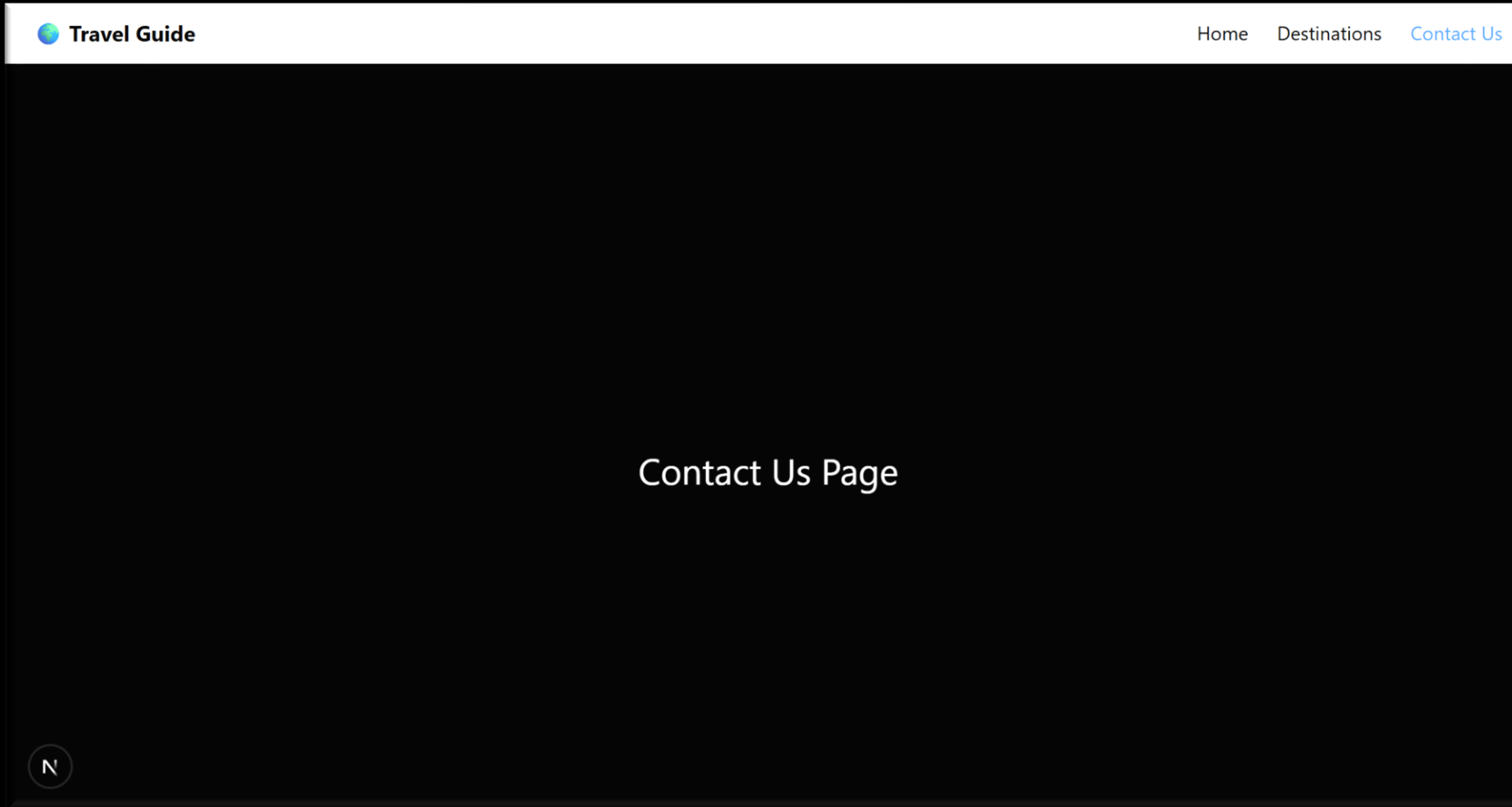




Practice Project 1

VIRTUAL CODE

Build a simple Travel Guide website using next JS -



ULTIMATE NEXT JS COURSE



LEVEL 4

Typescript Tutorial





What is TypeScript ?

- TypeScript is a strongly typed, object-oriented, compiled programming language developed by Microsoft.
- It is a superset of JavaScript, meaning that all valid JavaScript code is also valid TypeScript code.
- The primary addition TypeScript brings to JavaScript is static typing, allowing developers to define the types of variables, function parameters, and return values.



Basic Types -

Number

```
let age: number = 25;
```

String

```
let name: string = "Aman";
```

Boolean

```
let isOnline: boolean = true;
```

Array

```
let scores: number[] = [10, 20, 30];
```

Tuple

```
let user: [string, number] = ["Aman", 23];
```

Enum

```
enum Role { Admin, User, Guest }
```

Any

```
let something: any = "can be anything";
```

Void

```
function log(): void { console.log("Hi"); }
```

Null / Undefined

```
explicit: let x: null = null;
```



Arrays & Tuples -

💡 Arrays

ts

```
let scores: number[] = [10, 20, 30];  
let names: Array<string> = ["Aman", "Ravi"];
```

💡 Tuples

Fixed-length, ordered with specific types.

ts

```
let user: [string, number] = ["Aman", 22];
```



Functions -

💡 Basic Functions

ts

```
function add(a: number, b: number): number {  
    return a + b;  
}
```

💡 Optional & Default Parameters

ts

```
function greet(name: string, prefix: string = "Hi"): string {  
    return `${prefix}, ${name}`;  
}
```



Type Declaration & Type Inference -

Explicit Declaration

ts

```
let count: number = 5;
```

Type Inference

ts

```
let age = 20;           // inferred number
let city = "Delhi";     // inferred string
function double(x: number) {
  return x * 2;         // inferred number
}
```




Type Aliases -

Reusable type definitions:

ts

```
type ID = number | string;
type Status = "success" | "error" | "loading";

let userId: ID = 42;
let state: Status = "success";
```

Also for function signatures:

ts

```
type MathFn = (a: number, b: number) => number;
const divide: MathFn = (a, b) => a / b;
```



Interfaces -

```
ts

interface User {
  id: number;
  name: string;
  email?: string;    // optional
  readonly role: string;
}

const u: User = { id: 1, name: "Aman", role: "Admin" };
```

Extend:

```
ts

interface Employee extends User {
  salary: number;
}
```



Difference b/w Type & Interface -

Feature	type	interface
Combine	<code>&</code>	<code>extends</code>
Merge	✗	✓
Union	✓	✗
Use-case	primitives, unions, functions	object shapes

ts

```
type A = { a: number };  
type B = { b: string };  
type AB = A & B;
```

```
interface X { x: number; }  
interface Y extends X { y: string; }
```



Union & Intersection -

ts

// Union

```
type Response = "ok" | "fail" | "loading";
```

```
let res: Response = "ok";
```

// Intersection

```
type Person = { name: string };
```

```
type Contact = { phone: string };
```

```
type Full = Person & Contact;
```

```
const user: Full = { name: "Aman", phone: "123" };
```



Generics -

Reusable, type-safe structures.

ts

```
function identity<T>(value: T): T {  
    return value;  
}  
  
const n = identity(10);  
const s = identity("hello");  
  
// Generic Interface  
interface Box<T> { value: T; }  
const stringBox: Box<string> = { value: "Hi" };
```



Global Declaration -

src/types.d.ts

```
ts

declare global {
  interface Window { appVersion: string; }

  type ApiResponse<T> = {
    data: T;
    error?: string;
  };
}
export {};
```



React with TS -

Props

tsx

```
type ButtonProps = {  
  label: string;  
  onClick: () => void;  
};  
  
const Button: React.FC<ButtonProps> = ({ label, onClick }) =>  
  <button onClick={onClick}>{label}</button>;
```

useState / useRef

tsx

```
const [count, setCount] = useState<number>(0);  
const inputRef = useRef<HTMLInputElement>(null);
```



Type	Description	Common Use
<code>ReactNode</code>	Anything React can render	Children props
<code>ReactElement</code>	A single React element instance	One element prop
<code>JSX.Element</code>	Return type of component	Function component return
<code>ReactChild</code>	string, number, <code>ReactElement</code>	Strict single child
<code>ReactPortal</code>	For portals	Portal rendering
<code>ReactFragment</code>	For <code><>...</></code>	Grouping
<code>ReactComponentType<P></code>	Any component with props P	Dynamic rendering
<code>React.FC<P></code>	Function Component type	Components with children



✓ Best Practice Summary

1. `children` → use `React.ReactNode`
2. `element prop` → use `React.ReactElement`
3. `component return type` → use `JSX.Element`
4. `dynamic component as prop` → use `React.ComponentType<P>`
5. `multiple possible types` → use `ReactNode | null`

ULTIMATE NEXT JS COURSE



LEVEL 5

API Routes In NEXT JS



What is an API Route?

- An API Route is a **backend function** inside your Next.js app.
- It can **receive requests** and **send responses** (like a mini server).
- You don't need a separate Node/Express server.

bash

`/app/api/`

- Each folder can have a `route.js` (or `route.ts`) file.
- Each `route.js` = one API endpoint.

Example:

swift

`/app/api/hello/route.js`

URL → `/api/hello`



API Routes in Next JS

VIRTUAL CODE

1 Simple GET API → /api/hello

```
ts

// app/api/hello/route.ts
import { NextResponse } from "next/server";

export async function GET() {
  return NextResponse.json({ message: "Hello from TypeScript API!" });
}
```

2 POST API → /api/submit

```
ts

// app/api/submit/route.ts
import { NextRequest, NextResponse } from "next/server";

export async function POST(request: NextRequest) {
  const body = await request.json();
  return NextResponse.json({ message: "Data received!", data: body });
}
```



API Routes in Next JS

VIRTUAL CODE

1 Dynamic Route with `NextRequest` → `/api/users/[id]`

ts

```
// app/api/users/[id]/route.ts
import { NextRequest, NextResponse } from "next/server";

interface Params {
  params: { id: string };
}

export async function GET(request: NextRequest, { params }: Params) {
  const { id } = params;

  // You can also access request info
  const userAgent = request.headers.get("user-agent");

  return NextResponse.json({ userId: id, userAgent });
}
```

✓ URL: `/api/users/123`

Response:

json

```
{ "userId": "123", "userAgent": "..." }
```



API Routes in Next JS

VIRTUAL CODE

2 Query Parameter Route with `NextRequest` → `/api/search?q=nextjs`

ts

```
// app/api/search/route.ts
import { NextRequest, NextResponse } from "next/server";

export async function GET(request: NextRequest) {
  // Access query parameter ?q=nextjs
  const query = request.nextUrl.searchParams.get("q");

  // Access headers if needed
  const userAgent = request.headers.get("user-agent");

  return NextResponse.json({ query, userAgent });
}
```

✓ URL: `/api/search?q=nextjs`

Response:

json

```
{ "query": "nextjs", "userAgent": "..." }
```

ULTIMATE NEXT JS COURSE



LEVEL 6

Data Fetching in NEXT JS



Data Fetching in NEXT JS

VIRTUAL CODE

Data Fetching ka matlab hai:

“Server ya API se data laana taaki page render ho sake.”

Next.js me data fetching ke 3 major tareeke hote hain (App Router ke context me):

Type	Full Form	Kab Data Fetch Hota Hai	Use Case
SSR	Server Side Rendering	Har request pe	Dynamic pages (dashboard, user profile)
SSG	Static Site Generation	Build time pe	Blogs, landing pages
ISR	Incremental Static Regeneration	Build time + background me re-generate	News, e-commerce, changing content



◆ Simple Analogy

Type	Example Analogy
SSR	Har baar taja khana banana 🔍
SSG	Ek baar bana ke store karna 📦
ISR	Store kiya hua khana, lekin har kuch der baad naya bana dena 🔄



Next.js 15 (App Router) me ye kaise karte hain?

Ab hum `getServerSideProps` ya `getStaticProps` nahi likhte.

Uski jagah `fetch()` ke options se control karte hain:

Type	fetch Option
SSR	<code>{ cache: 'no-store' }</code>
SSG	<code>{ cache: 'force-cache' }</code> (default)
ISR	<code>{ next: { revalidate: x } }</code>



Data Fetching in NEXT JS

VIRTUAL CODE

● (A) SSR – Server Side Rendering

Har request pe fresh data laata hai

📁 app/ssr/page.js

jsx

📄 Copy code

```
export default async function SSRPage() {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts/1', {
    cache: 'no-store', // disables caching
  });
  const data = await res.json();

  return (
    <div>
      <h1>Server Side Rendering (SSR)</h1>
      <p>{data.title}</p>
      <p>Rendered at: {new Date().toLocaleTimeString()}</p>
    </div>
  );
}
```

📖 Explanation:

- `cache: 'no-store'` → har request pe API call hoti hai.
- Page refresh karne pe **time badalta dikhega** → proof that it's SSR.



Data Fetching in NEXT JS

VIRTUAL CODE

● (B) SSG – Static Site Generation

Build time pe data fetch hota hai (ek hi baar)

📁 app/ssg/page.js

jsx

📄 Copy code

```
export default async function SSGPage() {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts/2', {
    cache: 'force-cache', // default behavior
  });
  const data = await res.json();

  return (
    <div>
      <h1>Static Site Generation (SSG)</h1>
      <p>{data.title}</p>
      <p>Built at: {new Date().toLocaleTimeString()}</p>
    </div>
  );
}
```

📖 Explanation:

- Ye page **build ke time** render hota hai.
- Refresh karne pe **same time** rahega (static content).



Data Fetching in NEXT JS

VIRTUAL CODE

● (C) ISR – Incremental Static Regeneration

Static page banta hai, lekin background me auto refresh hota hai

app/isr/page.js

jsx

Copy code

```
export default async function ISRPage() {
  const res = await fetch('https://jsonplaceholder.typicode.com/posts/3', {
    next: { revalidate: 10 }, // 10 seconds ke baad background me update
  });
  const data = await res.json();

  return (
    <div>
      <h1>Incremental Static Regeneration (ISR)</h1>
      <p>{data.title}</p>
      <p>Generated at: {new Date().toLocaleTimeString()}</p>
    </div>
  );
}
```

📖 Explanation:

- Ye page static hai, lekin **10 seconds** ke baad Next.js background me naya version bana lega.
- Refresh karte raho — 10 sec baad "Rendered at" time change ho jaayega (auto regeneration).



3. Summary Table

Concept	Data Fetch Time	fetch Option	Updates?	Example Use
SSR	On every request	<code>{ cache: 'no-store' }</code>	Every reload	Dashboard, profile
SSG	At build time	<code>{ cache: 'force-cache' }</code> <code>}</code>	Never	Blogs, docs
ISR	Build + background regenerate	<code>{ next: { revalidate: x }</code> <code>} }</code>	Every X seconds	Product pages, news



Practice Project 2

VIRTUAL CODE

Build a Full Stack Next JS App in which –

- Use MongoDB Atlas 
- Authentication Using Next Auth 
- Image Upload Using Cloudinary 



Practice Project 2

VIRTUAL CODE

MongoDB Atlas 

Important Note for Creating Model-

"Ab guys yaha ek chhoti si line likhte hain, lekin ye line beginners ko confuse kar deti hai —

```
const User = mongoose.models?.User || mongoose.model("User", userSchema)
```

Ye hum karte kyun hain? Direct `mongoose.model("User", userSchema)` kyu nahi likhte?

Chalo easy example se samajhte hain 🙏"



🌱 Step 1 – Mongoose model ka basic idea

“Dekho jab bhi hum Mongoose me koi model banate hain, jaise:

js

📄 Copy code

```
const User = mongoose.model("User", userSchema)
```

Toh Mongoose ke andar internally ek list hoti hai — ek registry — jisme wo sab models ko store karta hai. Matlab uske paas ek object hota hai jisme likha hota hai:

js

📄 Copy code

```
mongoose.models = { User: UserModel }
```

Ab agar tum dobara se wahi model banane ki koshish karoge, toh Mongoose bolega — ‘Bhai ye model to pehle se bana hua hai, dubara mat banao!’ 🙄”



MongoDB Atlas



⚙️ Step 2 – Next.js me problem kyu aati hai

“Ab ye problem Next.js me isliye aati hai kyunki Next.js ka backend part har request ya code change ke time

automatically reload ho jata hai.

Matlab agar tumne model import kiya hua hai, toh Next.js us file ko dobara run karega.

Pehli baar to model bana lega,

lekin second time jab Next.js reload karega — toh same model dobara banane ki koshish karega, aur error aayega 🙅

```
OverwriteModelError: Cannot overwrite 'User' model once compiled.”
```



🧠 Step 3 – Iska perfect solution

“Ab is error se bachne ke liye hum kya karte hain?

Hum check kar lete hain — agar model pehle se bana hua hai to wahi use karo, nahi to naya banao.

Code likhte hain 📌

js

📄 Copy code

```
const User = mongoose.models?.User || mongoose.model("User", userSchema);  
export default User;
```

Yahaan `mongoose.models?.User` ka matlab hai:

pehle se koi `User` model hai kya?

Agar haan → use karo.

Agar nahi → naya model banao.”



MongoDB Atlas 

- How to Connect MongoDB in Next JS

Step 1 – Basic idea

“Sabse pehle samjho — Next.js ek full-stack framework hai.
Matlab ye frontend ke saath backend APIs bhi handle karta hai.

Jab hum backend part me MongoDB se connect karte hain,
to hume `mongoose.connect()` call karna padta hai.

Lekin Next.js me ek problem aati hai —
wo hot reload karta hai.

Har baar file change hone ya new API call pe backend dobara run hota hai,
aur agar humne seedha `mongoose.connect()` likh diya to
har baar ek naya database connection banega ❌”



- How to Connect MongoDB in Next JS

⚠ Step 2 – Problem samjho (Visual Example)

“Maan lo tumhare paas ek API route hai `/api/user` .
Har request ke time Next.js backend ko reload karta hai
aur tumhara code fir se run hota hai.
Matlab agar tumne likha:

```
js
```

```
mongoose.connect(MONGO_URL)
```

to ye line har baar execute hogi 🔄

Result: MongoDB me multiple open connections

→ app slow, crash, aur warning ‘Too many connections’.”



- How to Connect MongoDB in Next JS

💡 Step 3 – Solution idea (Connection reuse)

“Ab iska simple solution ye hai —
Ek baar connection bana lo,
aur usko **cache** me store kar lo.

Agli baar jab code chale to check karo:

- “Agar pehle se connection hai → use karo ✅”
- “Agar nahi hai → naya banao 🔄”

Ye caching hum **global object** me karte hain.”



- How to Connect MongoDB in Next JS

✖ Step 4 – `global` kya hota hai?

“Ab ye ‘global’ kya hota hai?

Simple language me samjho:

Browser me hota hai `window`,

aur Node.js (Next.js ke backend me) hota hai `global`.

Ye ek object hai jisme tum data store kar sakte ho
jo poore server ke lifetime tak rahega.

Matlab:

```
js
```

```
global.name = "Ankush"  
console.log(global.name) // Ankush
```

Ab chahe tum kitni bhi files change karo,
ye `global` object tab tak rahega jab tak server band nahi hota.”



Practice Project 2

VIRTUAL CODE

- How to Connect MongoDB in Next JS

"Ab agar aap TypeScript use kar rahe ho,
to TypeScript bolega:
'Property mongoose does not exist on globalThis.'
Matlab usse ye `global.mongoose` samajh nahi aa raha."

ts

```
import { Connection } from "mongoose";

declare global {
  var mongoose: {
    conn: Connection | null;
    promise: Promise<Connection> | null;
  };
}

export {};
```




- How to Connect MongoDB in Next JS

Explanation:

Line	Meaning
<code>declare global</code>	TypeScript ko bol rahe hain ki global scope me ek variable hai
<code>var mongoose: {...}</code>	us variable ka type define kar rahe hain
<code>Connection</code>	Mongoose ka actual connection type
<code>export {}</code>	File ko module banata hai taaki ye declaration valid rahe

"Ye sirf TypeScript ke liye hota hai,
runtime me ye code execute nahi hota.
Ye TypeScript ko bata deta hai ki
`global.mongoose` ek object hai jisme
`conn` aur `promise` properties hain."



Authentication Using Next Auth



1 What is NextAuth.js?

NextAuth.js ek authentication library hai Next.js ke liye.

Yeh login / signup system ko handle karta hai bina manually JWT, cookies, sessions likhe.

Example:

- User Google / GitHub / Facebook se login kar sakta hai (OAuth).
- Ya simple email-password (Credentials login) se bhi.
- Ye user ko session (token) ke form me track karta hai.
- Frontend & backend dono me secure rehta hai.



Practice Project 2

VIRTUAL CODE

Authentication Using Next Auth



User -> /api/auth/signin (NextAuth route)



Provider (Google/Credentials)



JWT Token Generate



Session Create Hoti Hai



Frontend: useSession() se access karte ho



Practice Project 2

VIRTUAL CODE

Authentication Using Next Auth



User -> /api/auth/signin (NextAuth route)



Provider (Google/Credentials)



JWT Token Generate



Session Create Hoti Hai



Frontend: useSession() se access karte ho



Practice Project 2

VIRTUAL CODE

Authentication Using Next Auth



Step 1: Install dependencies

```
bash
```

```
npm install next-auth bcryptjs mongoose
```



Practice Project 2

VIRTUAL CODE

Authentication Using Next Auth



Step 2: Create a route for NextAuth

```
/app/api/auth/[...nextauth]/route.ts
```

ts

```
import NextAuth from "next-auth";  
import authOptions from "@lib/auth";  
  
const handler = NextAuth(authOptions);  
  
export { handler as GET, handler as POST };
```

💡 Ye NextAuth ka "entry point" hai —

`authOptions` ke andar sari configuration hoti hai (providers, callbacks, etc).



What we write in authOptions ?

Ye NextAuth ka main configuration object hai —

isme hum batate hain:

- kaunse provider (Google / Credentials),
- callback kaise chalega,
- session kaise store hoga,
- custom pages kaha hain,
- secret key kya hai, etc.



Practice Project 2

VIRTUAL CODE

What we write in authOptions ?

Structure of authOptions

ts

```
const authOptions: NextAuthOptions = {  
  providers: [...],  
  callbacks: {...},  
  pages: {...},  
  session: {...},  
  secret: "...",  
};
```




Practice Project 2

VIRTUAL CODE

What we write in authOptions ?

Structure of authOptions

ts

```
const authOptions: NextAuthOptions = {  
  providers: [...],  
  callbacks: {...},  
  pages: {...},  
  session: {...},  
  secret: "...",  
};
```



Practice Project 2

VIRTUAL CODE

1. Providers option

Providers ka matlab:

“User kaise login karega?”

User fills form → authorize() → DB verify → Return user → JWT banega



1. Providers option

```
import CredentialsProvider from "next-auth/providers/credentials";
...
providers: [
  CredentialsProvider({
    // The name to display on the sign in form (e.g. "Sign in with...")
    name: "Credentials",
    // `credentials` is used to generate a form on the sign in page.
    // You can specify which fields should be submitted, by adding keys to the `credentials` object.
    // e.g. domain, username, password, 2FA token, etc.
    // You can pass any HTML attribute to the <input> tag through the object.
    credentials: {
      username: { label: "Username", type: "text", placeholder: "jsmith" },
      password: { label: "Password", type: "password" }
    },
    async authorize(credentials, req) {
      // Add logic here to look up the user from the credentials supplied
      const user = { id: "1", name: "J Smith", email: "jsmith@example.com" }

      if (user) {
        // Any object returned will be saved in `user` property of the JWT
        return user
      } else {
        // If you return null then an error will be displayed advising the user to check their details.
        return null

        // You can also Reject this callback with an Error thus the user will be sent to the error page with the
      }
    }
  })
]
...
```



Practice Project 2

VIRTUAL CODE

◆ (B) Google Provider (OAuth Login)

Google Provider ke liye bas clientId aur secret chahiye:

ts

```
GoogleProvider({  
  clientId: process.env.GOOGLE_CLIENT_ID!,  
  clientSecret: process.env.GOOGLE_CLIENT_SECRET!,  
})
```

💡 Flow:

pgsql

User clicks "Login with Google"

↓

Google Auth page open

↓

Google verify karta hai user ko

↓

Google returns email, name, image

↓

NextAuth callback me save ho jata hai

Is case me hum `authorize()` likhne ki zarurat nahi,
kyunki Google already user verify kar chuka hota hai.



2 Callbacks

Callbacks wo jagah hain jaha tum NextAuth ke flow ko control kar sakte ho.

- JWT Callback

- Ye chalta hai jab token generate hota hai.
- Hum isme custom fields add kar sakte hain (jaise `token.id`).

ts

```
async jwt({ token, user }) {  
  if (user) {  
    token.id = user.id;  
    token.email = user.email;  
  }  
  return token;  
}
```



Ye JWT me user data store karta hai (server side).



2 Callbacks

Callbacks wo jagah hain jaha tum NextAuth ke flow ko control kar sakte ho.

- JWT Callback

- Ye chalta hai jab token generate hota hai.
- Hum isme custom fields add kar sakte hain (jaise `token.id`).

ts

```
async jwt({ token, user }) {  
  if (user) {  
    token.id = user.id;  
    token.email = user.email;  
  }  
  return token;  
}
```



Ye JWT me user data store karta hai (server side).



Practice Project 2

VIRTUAL CODE

- session Callback

- Ye chalta hai jab frontend me `useSession()` call hota hai.
- Ye token se user data session me daalta hai.

ts

```
async session({ session, token }) {  
  if (session.user) {  
    session.user.id = token.id;  
    session.user.email = token.email;  
  }  
  return session;  
}
```

💡 Ye frontend ko final `session.user` object deta hai.



3

Session Object

ts

```
session: {  
  strategy: "jwt", // "jwt" = token based session (recommended)  
  maxAge: 30 * 24 * 60 * 60, // 30 days  
},
```

- "jwt" strategy ka matlab: har session me cookie ke andar JWT hota hai.
- "maxAge" se session expiry set hoti hai.



Custom Pages

ts

```
pages: {  
  signIn: "/login",  
  error: "/login",  
}
```

- Agar login page tumhara custom hai (/login),
to yahan specify kar dete ho.



Practice Project 2

VIRTUAL CODE



5

Secret

ts

```
secret: process.env.NEXTAUTH_SECRET,
```

- Ye JWT token ko encrypt karne ke liye use hota hai.
- Random strong key hona chahiye.



Practice Project 2

VIRTUAL CODE

- 1 User clicks "Login with Google" OR submits email/password
- 2 NextAuth -> Provider ke authorize() ya Google callback ko call karta hai
- 3 User verify hone ke baad -> jwt() callback
 - token me id, email, name set hota hai
- 4 Phir session() callback
 - token se session.user object banta hai
- 5 Frontend me useSession() call -> session.user milta hai
 - { id, name, email, image }
- 6 JWT cookie me store hota hai, automatic re-login possible hota hai



summary

“NextAuth ke andar sabse pehle hum GET aur POST export karte hain, jisse har request automatically handle ho jaye.
Phir hum authOptions define karte hain jaha providers hote hain — jaise Google OAuth ya apna custom credentials login.
Authorize function user ko verify karta hai,
JWT callback token create karta hai,
aur session callback us token ko frontend tak bhejta hai.
Is tarah humara full login flow secure aur simple ban jaata hai.”



summary

“NextAuth ke andar sabse pehle hum GET aur POST export karte hain, jisse har request automatically handle ho jaye.
Phir hum authOptions define karte hain jaha providers hote hain — jaise Google OAuth ya apna custom credentials login.
Authorize function user ko verify karta hai,
JWT callback token create karta hai,
aur session callback us token ko frontend tak bhejta hai.
Is tarah humara full login flow secure aur simple ban jaata hai.”